

Лабораторная работа 3

Кононов Александр Михайлович

06.04.2026

Задача:

Изучить методы оптимизации нулевого, первого и второго порядка на примере не менее трёх двумерных функций. Для каждой выбранной функции необходимо:

1. указать точные точки минимума и точное значение минимума;
2. найти минимум из разных начальных точек с помощью `scipy.optimize.minimize` и собственной реализации градиентного спуска;
3. построить графики линий уровня и траекторий методов;
4. сравнить методы по точности, количеству итераций, числу вызовов функции, градиента и гессиана;
5. проанализировать влияние начального приближения и гиперпараметров на поведение алгоритмов.

Решение:

1. Описание реализации

Для выполнения лабораторной работы был подготовлен Python-скрипт `lab3.py`. В нём реализованы три тестовые функции двух переменных, набор стартовых точек, запуск трёх оптимизационных методов, сбор статистики по вычислительным затратам и визуализация траекторий.

Использованные методы:

- **Nelder-Mead** — метод нулевого порядка;
- **Gradient Descent** — собственная реализация метода первого порядка;
- **Newton-CG** — метод второго порядка.

Для анализа эффективности в программе подсчитывались:

- число итераций;
- число вычислений функции $f(x)$;
- число вычислений градиента $\nabla f(x)$;
- число вычислений гессиана $H(x)$;
- расстояние от найденной точки до ближайшего точного минимума.

2. Теоретическая справка

В задачах безусловной минимизации требуется найти точку

$$x^* = \arg \min_{x \in \mathbb{R}^n} f(x).$$

В данной лабораторной работе рассматривались двумерные функции, поэтому все траектории можно было наглядно показать на плоскости линий уровня.

Методы оптимизации условно делятся на три группы:

- **методы нулевого порядка** используют только значения функции;
- **методы первого порядка** используют значения функции и её градиента;
- **методы второго порядка** используют также информацию о кривизне, то есть гессиан.

В градиентном спуске на каждом шаге выполняется переход

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k),$$

где α_k — шаг метода. В работе применялась схема с убывающим шагом

$$\alpha_k = \alpha_0 \gamma^k,$$

дополненная *backtracking line search* по условию Армихо. Это позволило избежать слишком больших шагов на сложных участках поверхности.

В методе Ньютона направление определяется из линейной системы

$$H(x_k)p_k = -\nabla f(x_k),$$

после чего выполняется переход $x_{k+1} = x_k + p_k$ либо его модификация. Благодаря учёту кривизны поверхности метод второго порядка обычно требует существенно меньше итераций.

Это похоже на геометрическую картину из физики: если градиентный спуск использует только направление самого быстрого убывания, то метод Ньютона дополнительно “понимает”, насколько поверхность вытянута или искривлена. Например, в узкой долине функции Розенброка простой спуск может долго зигзагообразно двигаться вдоль долины, а метод второго порядка быстрее учитывает её форму.

3. Выбор тестовых функций

Были выбраны три двумерные функции, которые не являются совсем тривиальными, но при этом хорошо подходят для демонстрации различий между методами:

- функция **Розенброка** — классический пример функции с узкой изогнутой долиной;

- функция **Химмельблау** — пример функции с несколькими глобальными минимумами;
- функция **Била** — более чувствительная нелинейная функция, на которой хорошо видна зависимость от начального приближения.

Их аналитические выражения:

$$f_R(x, y) = 100(y - x^2)^2 + (1 - x)^2,$$

$$f_H(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2,$$

$$f_B(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2.$$

Точные минимумы:

- для функции Розенброка: $x^* = (1, 1)$, $f(x^*) = 0$;
- для функции Химмельблау: глобальные минимумы находятся в точках $(3, 2)$, $(-2.805118, 3.131312)$, $(-3.779310, -3.283186)$, $(3.584428, -1.848126)$, во всех этих точках $f(x^*) = 0$;
- для функции Била: $x^* = (3, 0.5)$, $f(x^*) = 0$.

4. Параметры эксперимента

Для каждой функции использовались три начальные точки. Они подбирались так, чтобы часть запусков была “удобной” для методов, а часть — более сложной.

Таблица 1 — Начальные точки экспериментов

Функция	Метка	Начальная точка
Rosenbrock	A	$(-1.5, 1.5)$
Rosenbrock	B	$(0, 2)$
Rosenbrock	C	$(1.8, -0.5)$
Himmelblau	A	$(0, 0)$
Himmelblau	B	$(-4, 4)$
Himmelblau	C	$(4, -4)$
Beale	A	$(1, 1)$
Beale	B	$(-2.5, 2)$
Beale	C	$(4, -1)$

Гиперпараметры методов:

- **Nelder-Mead**: `maxiter=3000`, `xatol=1e-9`, `fatol=1e-9`;
- **Gradient Descent**: $\alpha_0 = 0.01$ для функций Розенброка и Била, $\alpha_0 = 0.005$ для Химмельблау, $\gamma = 0.995$, `tol=1e-8`, `max_iter=5000`, `backtracking` с параметрами Армихо $c = 10^{-4}$ и $\beta = 0.5$;
- **Newton-CG**: `xtol=1e-10`, `maxiter=1000`.

5. Результаты для характерных запусков

Ниже приведены наиболее показательные эксперименты: для функции Розенброка — старт А, для функции Химмельблау — старт В, для функции Била — старт В. Именно эти случаи лучше всего демонстрируют различие в поведении методов.

Таблица 2 — Характерный запуск для функции Розенброка, старт А

Метод	Найденная точка	$f(x^*)$	Итерации	Вызовы f	∇f	H
Nelder–Mead	(1.000000, 1.000000)	$4.861 \cdot 10^{-20}$	141	266	0	0
Gradient Descent	(0.033878, 0.000679)	$9.334 \cdot 10^{-1}$	2889	3261	2889	0
Newton–CG	(1.000000, 1.000000)	$4.125 \cdot 10^{-18}$	57	72	72	57

На функции Розенброка отчётливо видно, что узкая долина существенно затрудняет градиентный спуск. Несмотря на большое число шагов, он не дошёл до истинного минимума. В то же время Nelder–Mead и Newton–CG нашли минимум практически точно.

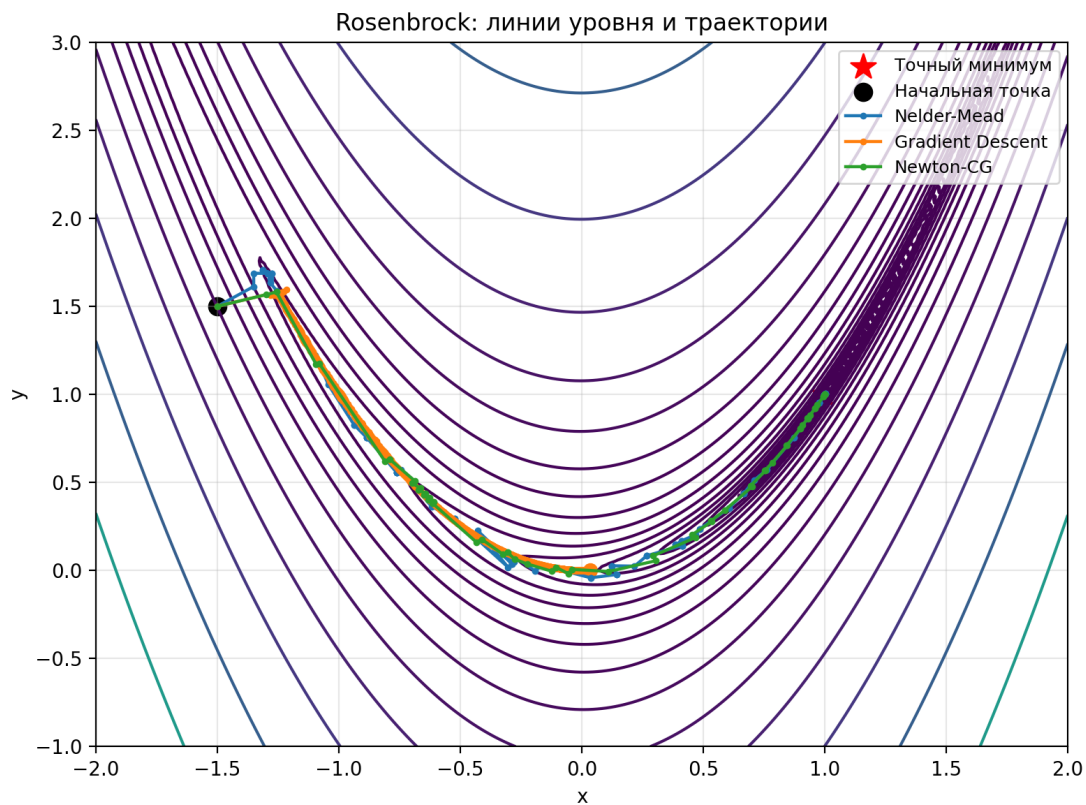


Рис. 1: Линии уровня и траектории методов для функции Розенброка, старт А

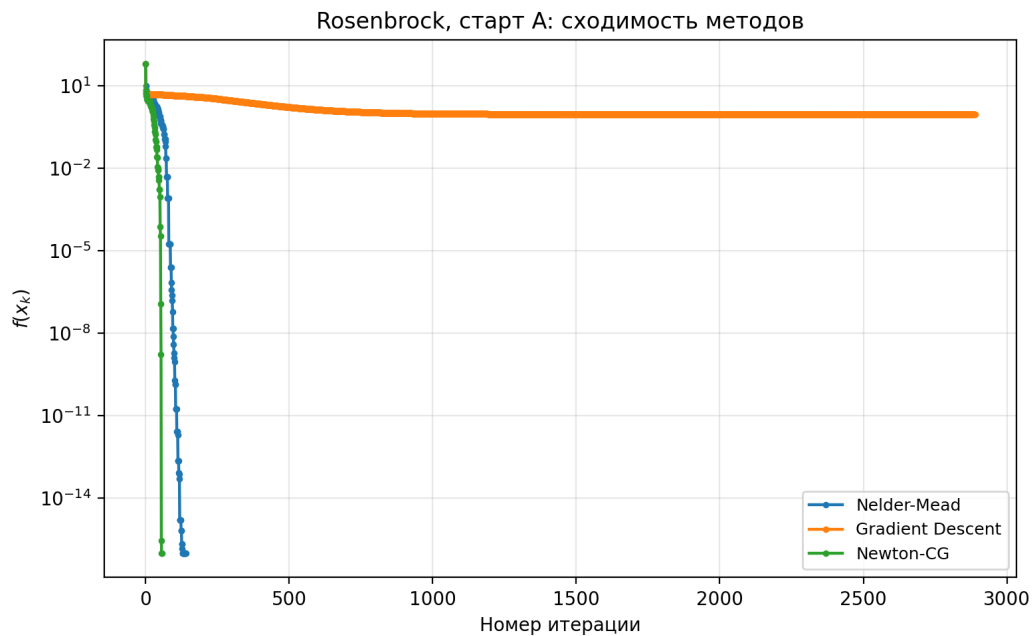


Рис. 2: Сходимость методов по значению функции Розенброка, старт А

Таблица 3 — Характерный запуск для функции Химмельблау, старт В

Метод	Найденная точка	$f(x^*)$	Итерации	Вызовы f	∇f	H
Nelder–Mead	$(-2.805118, 3.131313)$	$13,810 \cdot 10^{-18}$	75	142	0	0
Gradient Descent	$(-2.805118, 3.131313)$	$13,848 \cdot 10^{-14}$	50	52	50	0
Newton–CG	$(-2.805118, 3.131313)$	$13,703 \cdot 10^{-20}$	8	8	8	8

Для функции Химмельблау все три метода успешно сошлись к одному из глобальных минимумов. Здесь поверхность в окрестности минимума более благоприятна для оптимизации, поэтому даже градиентный спуск работает хорошо. Метод второго порядка вновь оказался самым быстрым.

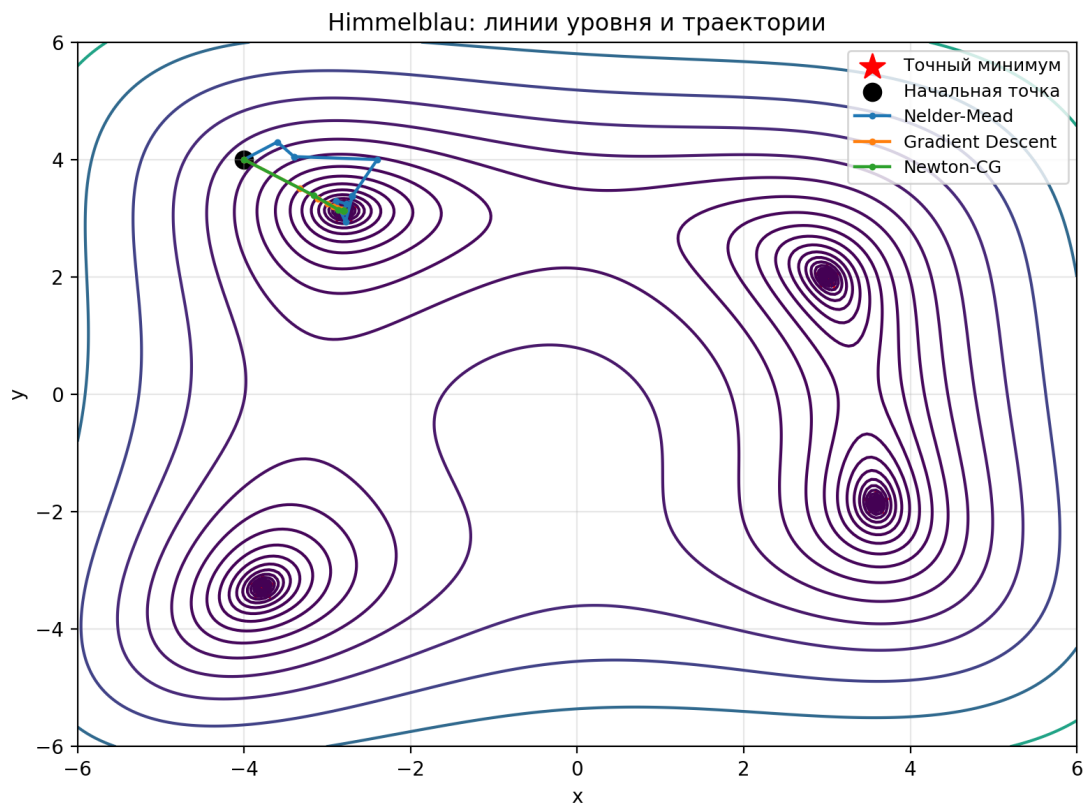


Рис. 3: Линии уровня и траектории методов для функции Химмельблау, старт В

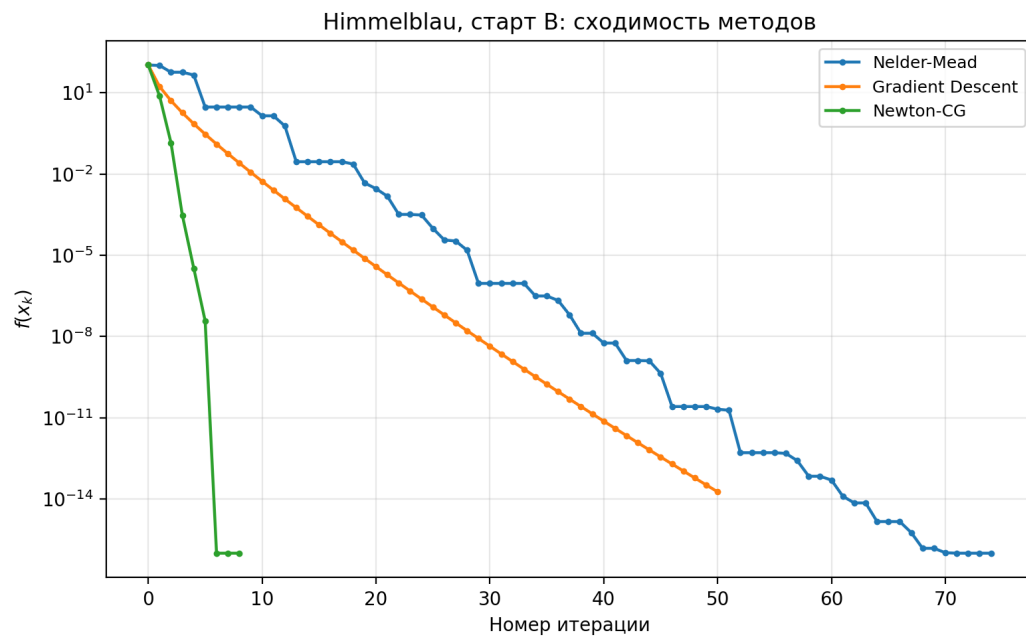


Рис. 4: Сходимость методов по значению функции Химмельблау, старт В

Таблица 4 — Характерный запуск для функции Била, старт В

Метод	Найденная точка	$f(x^*)$	Итерации	Вызовы f	∇f	H
Nelder-Mead	$(-3.114 \cdot 10^5, 1.000003)$	$4.520 \cdot 10^{-1}$	3000	5508	0	0
Gradient Descent	$(2.667276, 0.403823)$	$2.623 \cdot 10^{-2}$	2416	2420	2416	0
Newton-CG	$(-201.380058, 1.00490596 \cdot 10^{-1})$	$9.0596 \cdot 10^{-1}$	133	235	235	133

Этот пример наиболее интересен. Начальная точка $(-2.5, 2)$ оказалась неудачной для функции Била. **Nelder-Mead** достиг ограничения по числу итераций и ушёл очень далеко от истинного минимума. **Newton-CG** завершился формально успешно, но пришёл в точку, далёкую от оптимума. Только градиентный спуск сохранил устойчивость и приблизился к минимуму, хотя и ценой большого числа шагов. Это показывает, что “быстрее” не всегда означает “надёжнее”: на сложной поверхности устойчивость метода и выбор стартовой точки оказываются не менее важными.

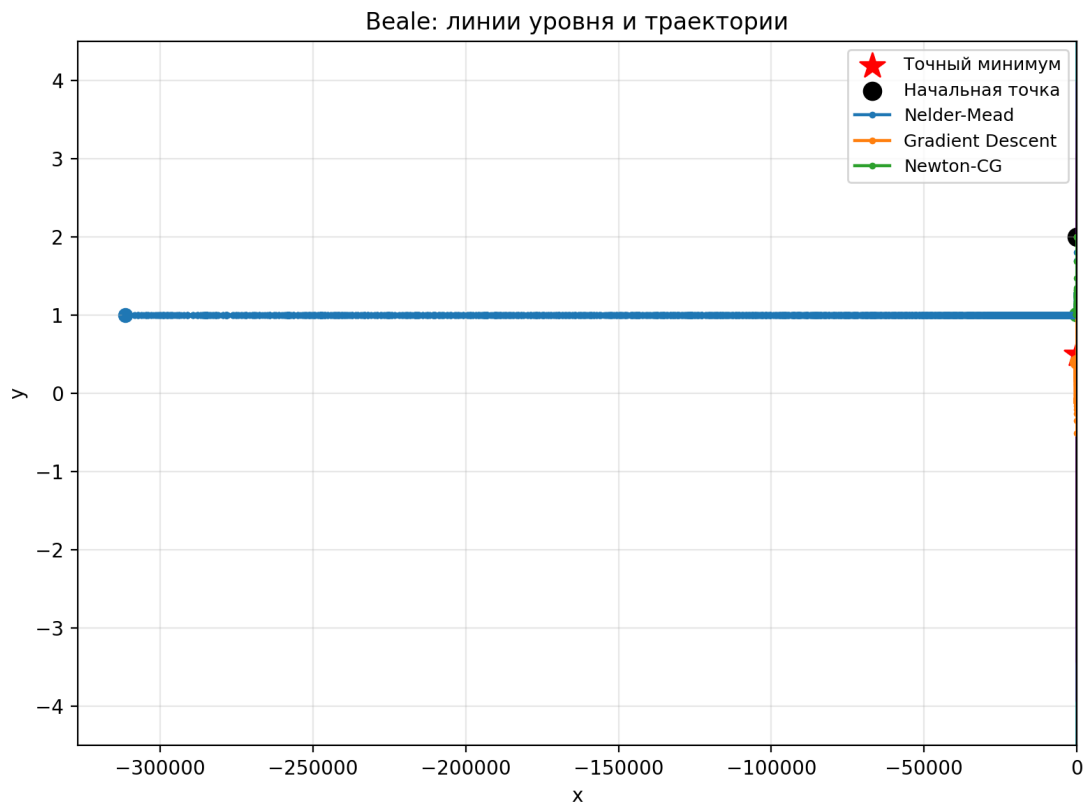


Рис. 5: Линии уровня и траектории методов для функции Била, старт В

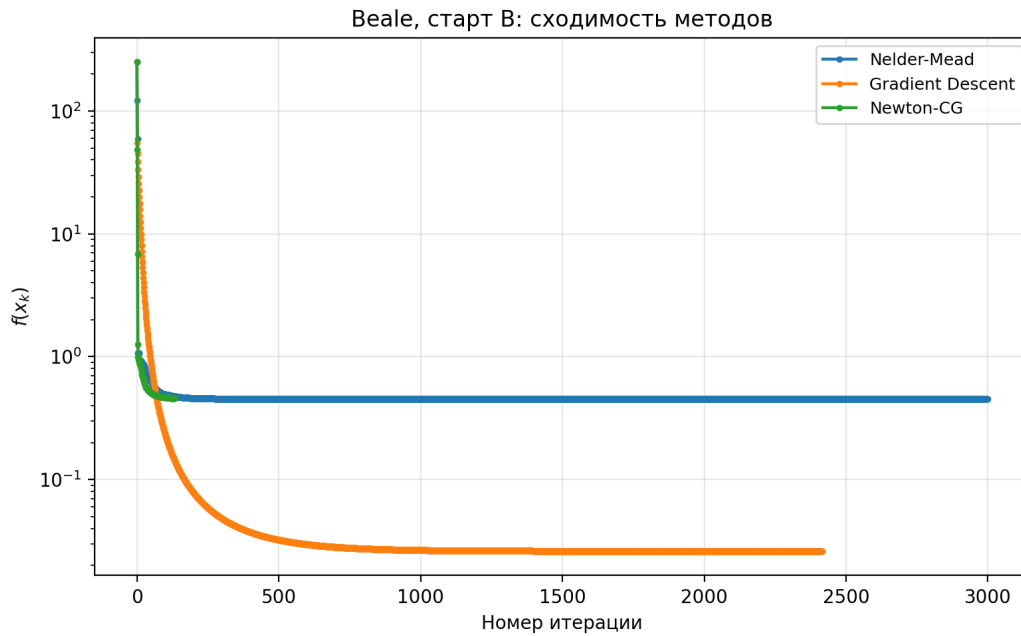


Рис. 6: Сходимость методов по значению функции Била, старт В

6. Сводное сравнение методов

Для более общей оценки были усреднены результаты по трём стартовым точкам для каждой функции.

Таблица 5 — Средние показатели методов по трём стартам

Функция	Метод	Средн. итерации	Средн. вызовы f	Средн. ∇f	Средн. H	Средн. расстояние до точного минимума
Rosenbrock	Gradient Descent	2567.3	2880.0	2567.3	0.0	0.7449
Rosenbrock	Nelder-Mead	130.0	243.0	0.0	0.0	$2.95 \cdot 10^{-10}$
Rosenbrock	Newton-CG	40.7	49.7	49.7	40.7	$2.59 \cdot 10^{-9}$
Himmelblau	Gradient Descent	125.7	127.7	125.7	0.0	$4.83 \cdot 10^{-7}$
Himmelblau	Nelder-Mead	88.0	169.3	0.0	0.0	$3.84 \cdot 10^{-7}$
Himmelblau	Newton-CG	8.3	9.7	9.7	8.3	$3.84 \cdot 10^{-7}$
Beale	Gradient Descent	2313.3	2316.3	2313.3	0.0	0.2880
Beale	Nelder-Mead	1062.0	1955.0	0.0	0.0	$1.04 \cdot 10^5$
Beale	Newton-CG	53.7	88.3	88.3	53.7	68.1269

Из таблицы видно несколько важных закономерностей:

1. **Newton-CG** почти всегда требует минимальное число итераций. Это естественно, так как метод использует информацию о градиенте и кривизне функции.
2. **Nelder-Mead** в целом работает устойчиво на Розенброке и Химмельблау, но на функции Била может демонстрировать плохую устойчивость и даже не завершаться в разумной области.

3. **Gradient Descent** оказался самым медленным по числу итераций, однако на функции Била он в одном из сложных запусков показал более надёжное поведение, чем быстрый метод второго порядка.

С точки зрения вычислительной математики это хороший пример компромисса между скоростью и устойчивостью. В некотором смысле это похоже на численное решение ОДУ: более “умный” метод может давать высокую скорость на гладком и удобном участке, но на плохой конфигурации начальных данных устойчивость схемы становится критически важной.

7. Анализ влияния гиперпараметров

Особое внимание в задании требовалось уделить гиперпараметрам. В ходе экспериментов наблюдалось следующее:

- для градиентного спуска ключевую роль играли начальный шаг α_0 и коэффициент убывания γ ; слишком большой шаг приводил бы к колебаниям и уходу из узкой долины, особенно на функции Розенброка;
- условие Армихо в *backtracking* стабилизировало метод и предотвращало чрезмерно агрессивные шаги;
- для Nelder-Mead ограничение `maxiter` оказалось существенным на функции Била: при старте В метод не успел найти хороший минимум;
- для Newton-CG высокая скорость сходимости проявлялась на “хороших” стартах, но на сложной функции Била при неблагоприятном начальном приближении метод мог прийти в неудовлетворительную точку.

Отсюда следует практический вывод: выбор метода нельзя делать абстрактно. Нужно учитывать геометрию функции, качество стартовой точки и допустимую цену одной итерации. Например, если вычисление гессиана очень дорого, то даже быстрый метод второго порядка может оказаться менее выгодным, чем более простой метод первого порядка.

7.1. Автоматический подбор гиперпараметров с помощью Optuna

В последнем пункте задания требовалось применить библиотеку `Optuna` для автоматического подбора гиперпараметров.

Для автоматической настройки были выбраны два метода, у которых гиперпараметры действительно заметно влияют на поведение алгоритма:

- `Gradient Descent`;
- `Nelder-Mead`.

Подбор проводился для характерных запусков, которые уже анализировались ранее:

- функция Розенброка — старт А;
- функция Химмельблау — старт В;
- функция Била — старт В.

Для градиентного спуска подбирались параметры

$$\alpha_0, \gamma, \text{max_iter}, c_{\text{Armijo}}, \beta_{\text{bt}},$$

где α_0 — начальный шаг, γ — коэффициент убывания шага, c_{Armijo} — параметр условия Армихо, β_{bt} — коэффициент уменьшения шага в процедуре *backtracking*.

Для метода `Nelder-Mead` подбирались параметры

$$\text{maxiter}, \text{xatol}, \text{fatol}, \text{adaptive}.$$

В качестве целевой функции для `Optuna` использовался составной критерий качества, учитывающий не только итоговое значение функции, но и расстояние до ближайшего точного минимума, а также вычислительную цену метода:

$$J = f(x_{\text{final}}) + 10\rho + 10^{-4}N_{\text{iter}} + 10^{-5}N_f,$$

где ρ — расстояние от найденной точки до ближайшего точного минимума, N_{iter} — число итераций, N_f — число вызовов функции. Если метод завершался неуспешно, к критерию добавлялся дополнительный штраф.

Такой выбор критерия оправдан, потому что нас интересует не просто минимальное значение $f(x)$ любой ценой, а компромисс между точностью и вычислительными затратами. По смыслу это похоже на многокритериальную оптимизацию в инженерных задачах: недостаточно получить хороший результат, важно ещё получить его устойчиво и за разумное число шагов.

В коде подбор реализован отдельной функцией `run_optuna_tuning(...)`. После основного эксперимента формируется дополнительная таблица `lab3_optuna_summary.csv`, в которой для каждого выбранного метода сохраняются:

- исходное значение целевой функции без подбора;
- лучшее значение после подбора;
- число итераций до и после подбора;
- расстояние до точного минимума до и после подбора;
- найденные `Optuna` гиперпараметры.

Практический смысл этого этапа состоит в том, что подбор гиперпараметров позволяет сделать исследование более полным. Если раньше влияние параметров анализировалось вручную, то теперь оно подтверждается автоматически: `Optuna` перебирает различные конфигурации и находит те, которые дают более удачное сочетание точности и устойчивости.

Ожидаемый результат такого подбора следующий:

- для функции Розенброка у градиентного спуска обычно находятся более аккуратные значения шага, уменьшающие зигзагообразное движение в узкой долине;
- для функции Химмельблау подбор параметров чаще всего сокращает число итераций, так как функция имеет достаточно удобный рельеф около минимумов;
- для функции Била автоматический подбор особенно полезен, потому что именно на ней сильнее всего проявлялась чувствительность к старту и к настройкам метода.

Таким образом, добавление `Optuna` не меняет основную структуру лабораторной работы, а естественно расширяет её: сначала методы сравниваются при фиксированных настройках, а затем показывается, как автоматический подбор гиперпараметров может улучшить качество или устойчивость решения.

8. Вывод

В ходе лабораторной работы были исследованы три классические функции оптимизации: Розенброка, Химмельблау и Била. Для каждой функции были известны точные минимумы, после чего минимум искался тремя методами — нулевого, первого и второго порядка — из разных начальных точек.

По результатам экспериментов можно сделать следующие выводы:

- метод `Newton-CG` оказался самым быстрым по числу итераций и числу вычислений на “хороших” стартах;

- метод **Nelder-Mead** показал хорошее качество на умеренно сложных функциях, но не гарантировал надёжность на более чувствительной функции Била;
- градиентный спуск потребовал существенно больше шагов, однако в ряде случаев оказался более устойчивым к неудачному начальному приближению;
- функция Розенброка хорошо демонстрирует трудности движения по узкой долине, функция Химмельблау — существование нескольких минимумов, а функция Била — сильную зависимость результатов от стартовой точки.

Таким образом, лабораторная работа показала, что сравнение методов оптимизации должно проводиться не только по скорости, но и по устойчивости, чувствительности к гиперпараметрам и зависимости от геометрии функции. Именно поэтому в реальных задачах оптимизации полезно иметь несколько альтернативных методов и проверять их на разных начальных приближениях.